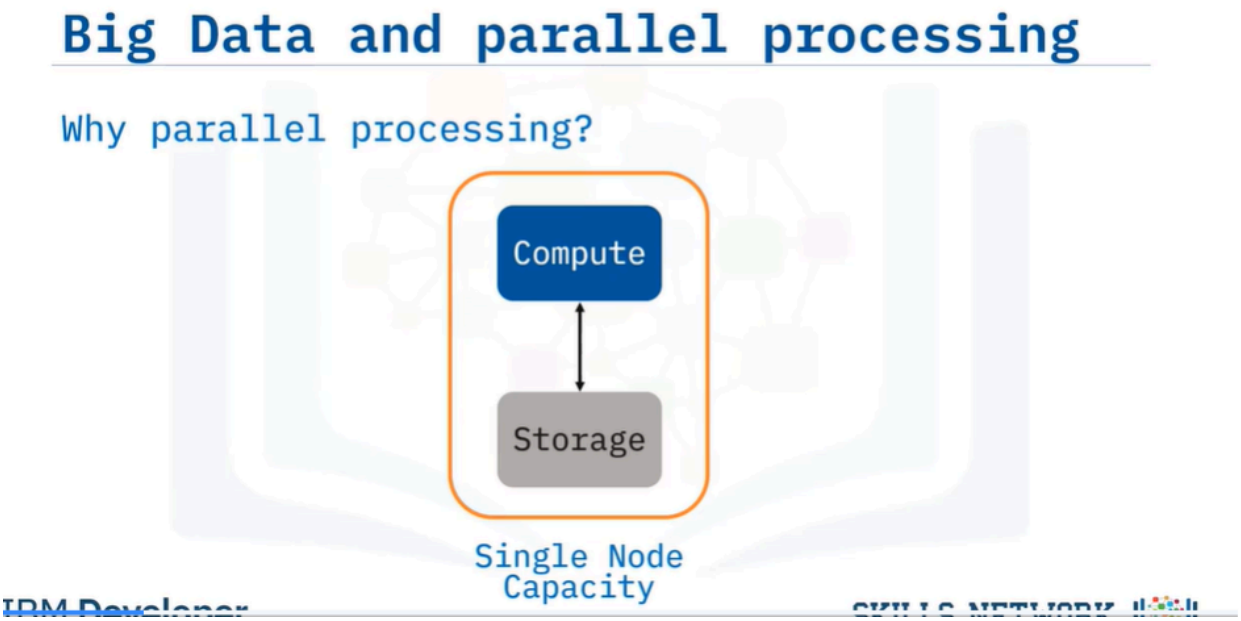
Parallel Processing, Scaling, and Data Parallelism

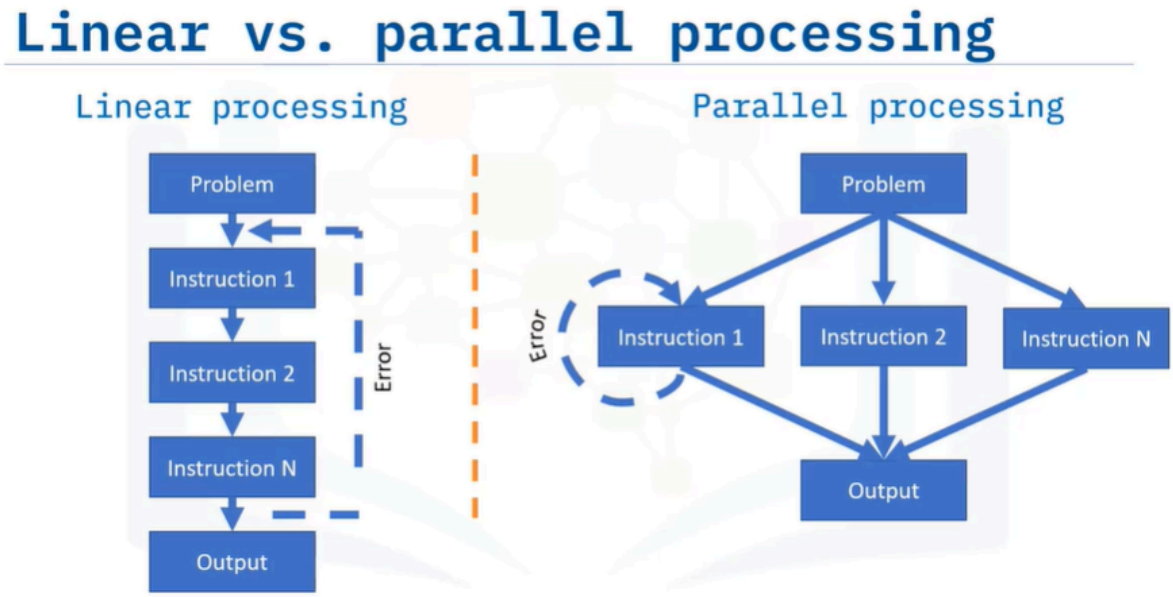
Sure! Let's talk about **Parallel Processing** in the context of Big Data.

Parallel processing is like having a team of chefs in a kitchen, each working on a different dish at the same time. Instead of one chef cooking a meal step by step, which can take a long time, multiple chefs can chop, sauté, and bake simultaneously. This teamwork allows them to prepare a large feast much faster than if they were working alone. In the world of computers, parallel processing means breaking down a big problem into smaller tasks and having multiple computers (or nodes) work on those tasks at the same time. This is especially useful for handling large amounts of data, known as Big Data, which is too much for a single computer to manage efficiently.

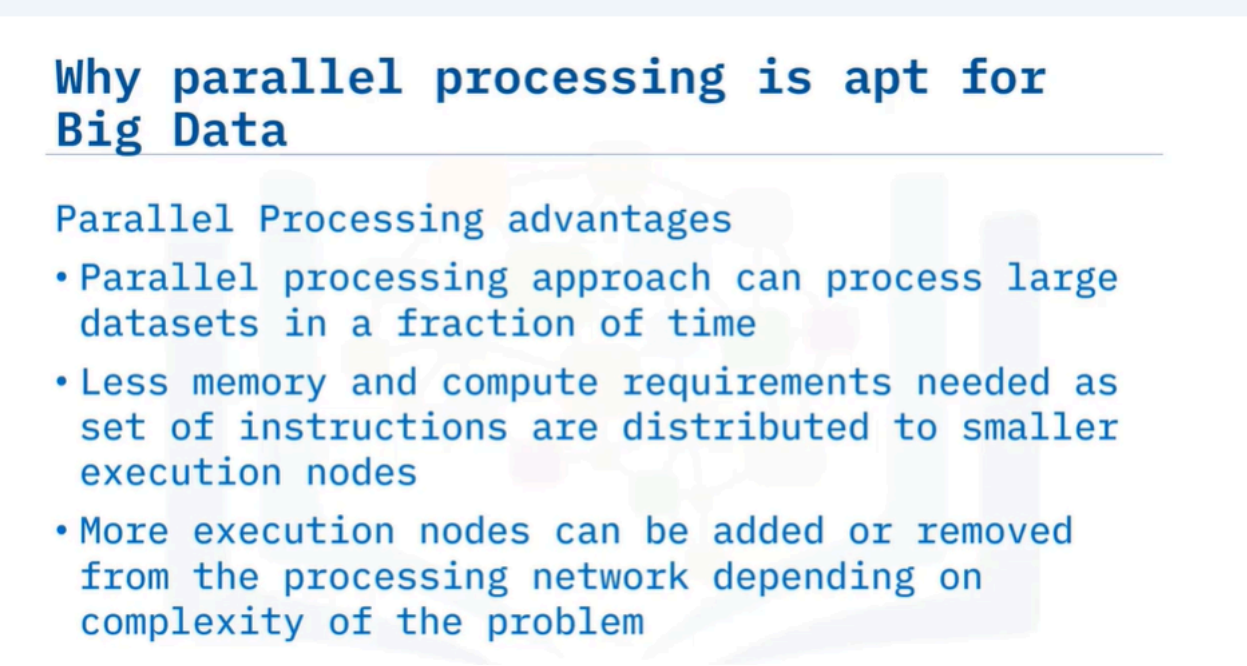
Now, imagine you have a huge library of books, and you want to find a specific quote. If you search through each book one by one, it could take forever. But if you have a group of friends helping you, each searching through different books at the same time, you'll find the quote much quicker! This is the essence of parallel processing—it speeds up the work by dividing it among many helpers, making it perfect for the vast amounts of data we deal with today.



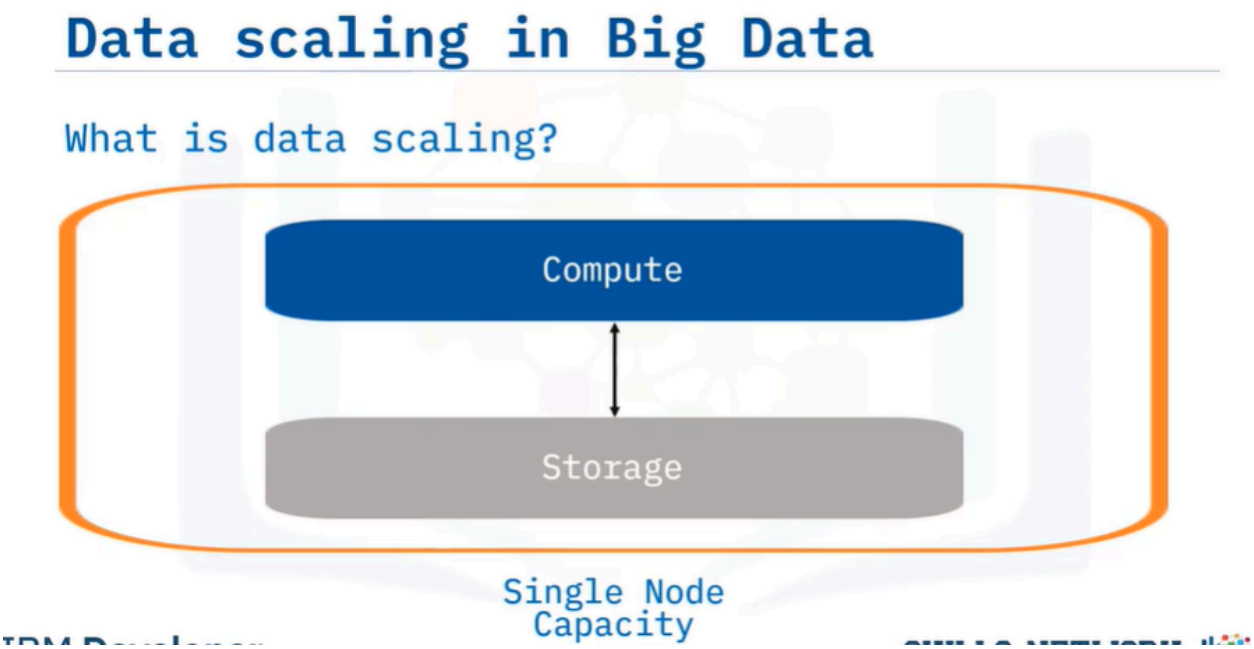
- You can do lots of things with data at various sizes, right? In any normal analytics cycle, the functionality of the computer is to store data and move that data from its storage capacity into a compute capacity (which includes memory), and back to storage once important results are computed.



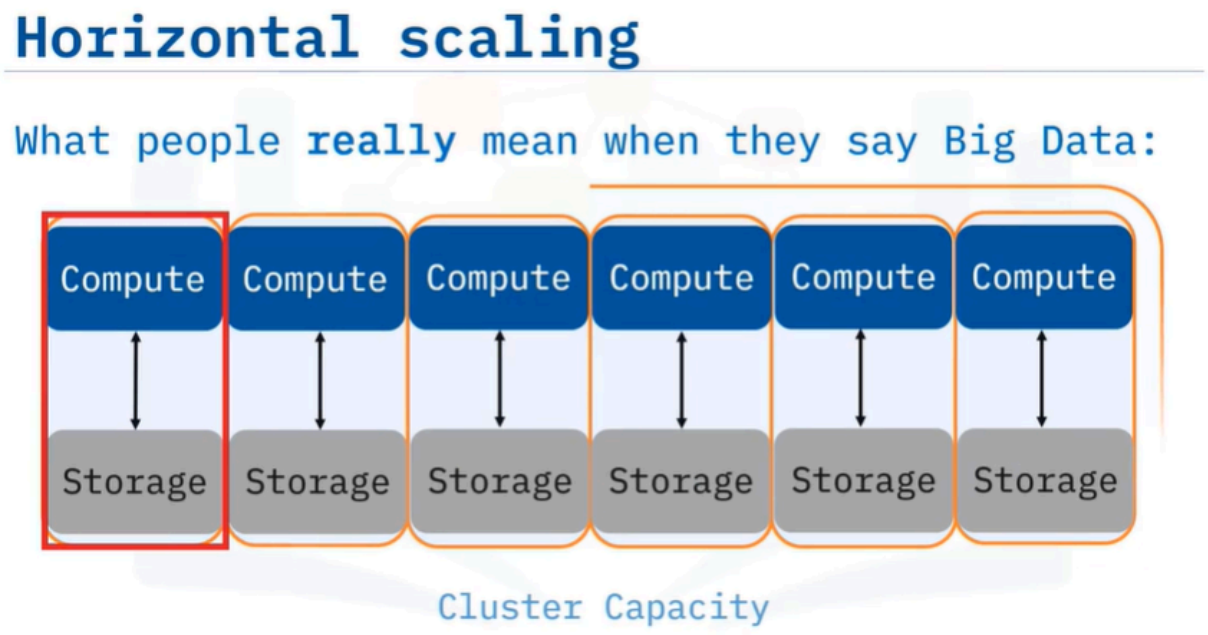
- With Big Data, you have more data than will fit on a single computer. Parallelism or Parallel processing can best be understood by comparing it to Linear processing. Linear Processing Linear processing is the traditional method of computing a problem where the problem statement is broken into a set of instructions that are executed sequentially till all instructions are completed successfully.
- If an error occurs in any one of the instructions, the entire sequence of instructions is executed from the beginning after the error has been resolved. It is evident from the processing method that Linear processing is best suited for minor computing tasks and is inefficient and time consuming when it comes to processing complex problems such as Big Data.
- Parallel Processing The alternative to Linear processing is parallel processing. Here too, the problem statement is broken down into a set of executable instructions. The instructions are then distributed to multiple execution nodes of equal processing power and are executed in parallel. Since the instructions are run on separate execution nodes, errors can be fixed and executed locally independent of other instructions.



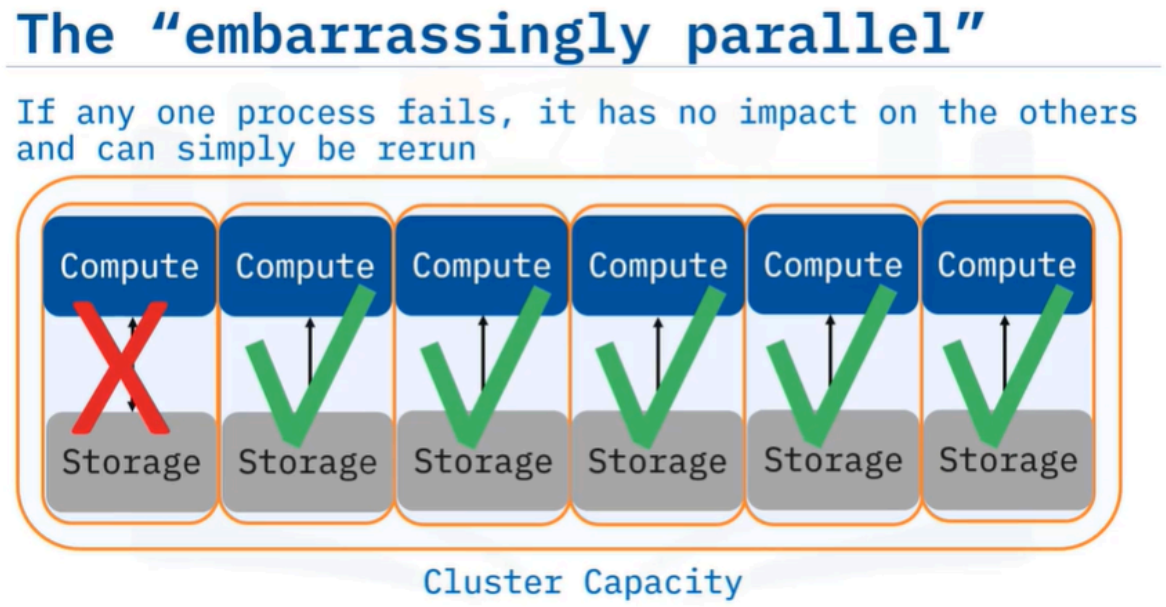
- As noted on the previous slide, Parallel processing offers significant advantages when dealing with complex problems such as Big Data. Some of the other benefits of using Parallel processing are:. Reduced processing times. Parallel processing can process Big Data in a fraction of the time compared to linear processing. Less memory and processing requirements. Since the problem instructions are executed on separate execution nodes, memory and processing requirements are low even while processing large volumes of data. Flexibility. The biggest advantage of parallel processing is that execution nodes can be added and removed as and when required. This significantly reduces infrastructure cost.



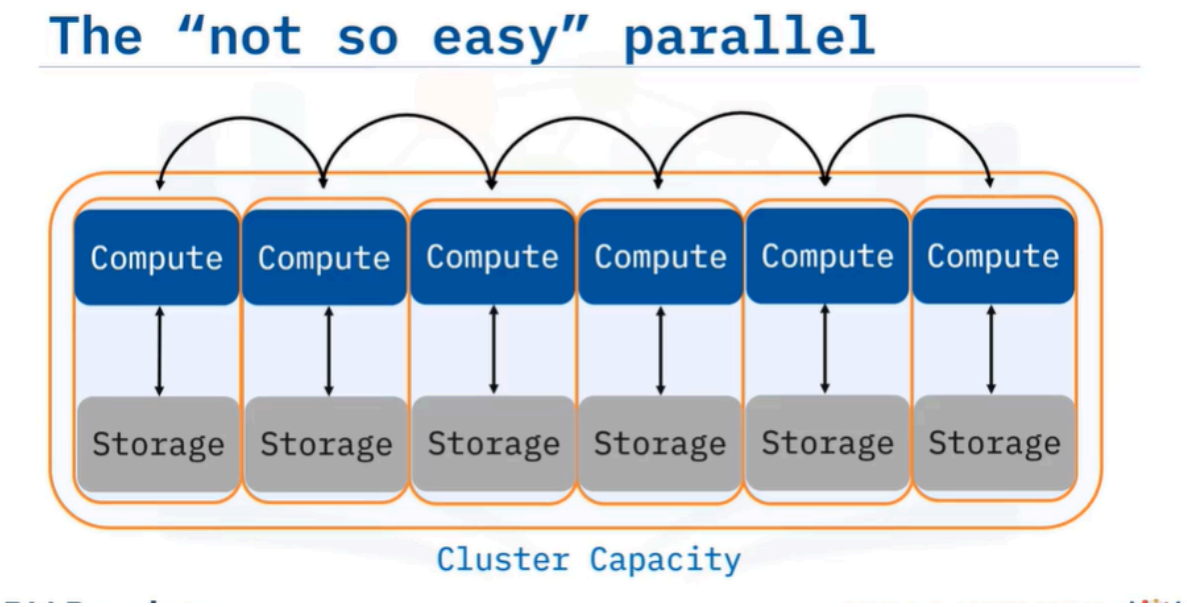
- Data Scaling is a technique to manage, store, and process the overflow of data. You can get a larger single node computer. But when your data is growing exponentially, eventually it will outgrow the capacity that is available. Increasing the capacity of a single node as a means of increasing capacity is called scaling up.



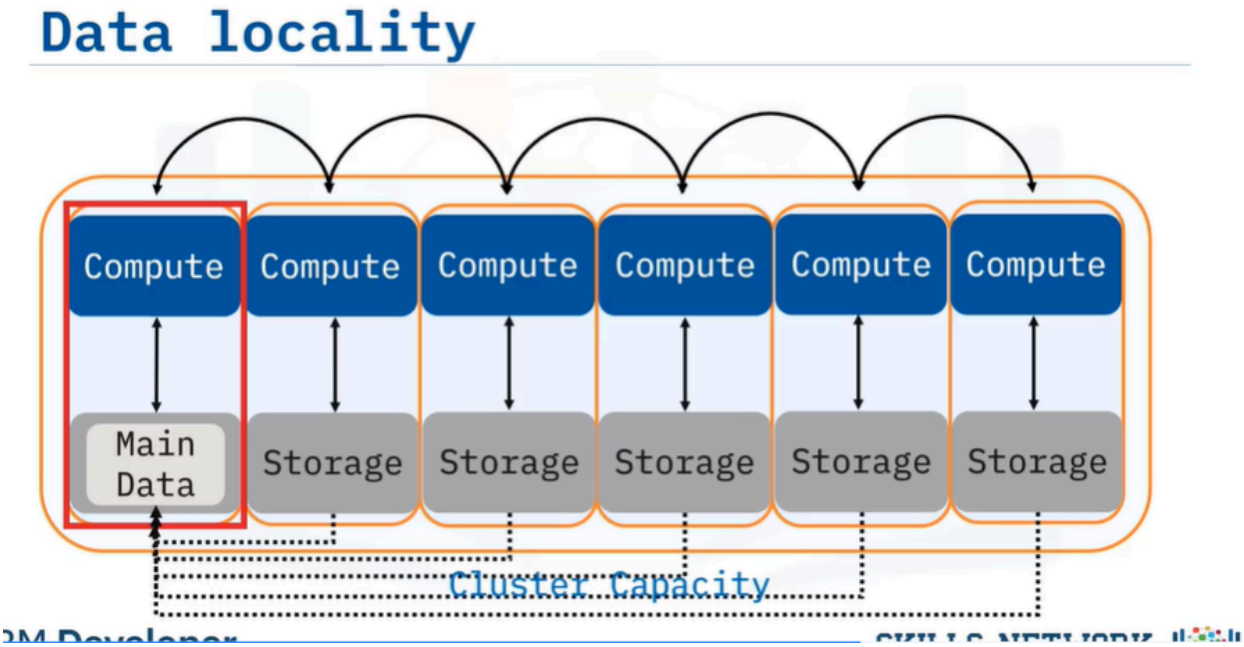
- A better strategy, or at least the one most people ultimately choose, is to scale out or to scale horizontally. This simply means adding additional nodes with the same capacity until the problem is tractable. The individual nodes arranged in this way are called a computing cluster.



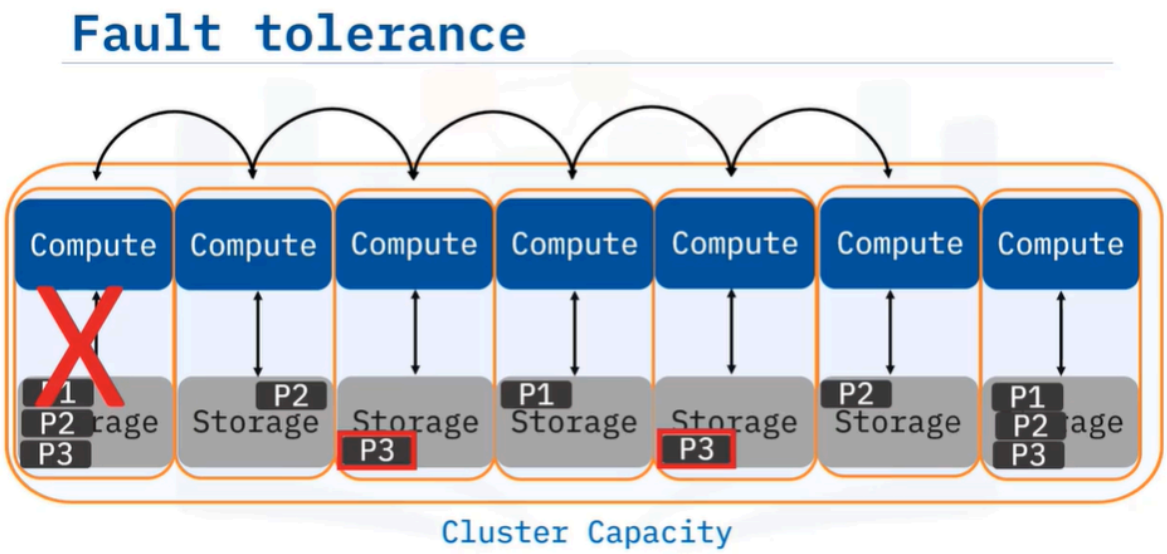
- Compute clusters can solve problems that are known as “embarrassingly parallel” calculations. These are the kind of workloads that can easily be divided and run independent of one another. If any one process fails, it has no impact on the others and can easily be rerun. An example would be to, say, change the date format in a single column of a large dataset that has been split into multiple smaller chunks that are stored in different nodes of the cluster.



- Sometimes, sorting a large data set adds significant complexity to the process. Now, the multiple computations must coordinate with one another because each process needs to be aware of the state of its peer processes in order to complete the calculation. This requires sending messages across a network to each other or writing them to a file system that is accessible to all processes on the cluster. The level of complexity increases significantly, because you are basically asking a cluster of computers to behave as a single computer.



- Most calculations in enterprise environments are considered “embarassingly parallel with some not so easy” calculations. While this statement is true to varying degrees it has guided the design of underlying frameworks. In the Hadoop ecosystem, the concept of “bringing compute to the data” is a central idea in the design of the cluster. The cluster is designed in a way that computations on certain pieces, or partitions, of the data will take place right at the location of the data when possible. The resulting output will also be written to the same node.



- Computers break, outages happen, and you need to be prepared. In such cases, fault tolerance comes into play. Fault tolerance refers to the ability of a system to continue operating without interruption when one or more of its components fail. This works for Hadoop primary data storage system (HDFS) and other similar storage systems (like S3 and object storage), a system like the one we showcase here.
- Consider the first 3 partitions of a data set labelled P1, P2, and P3, which reside on the first node. In this system, copies of each of these data partitions are also stored on other locations or nodes within the cluster.
- If the first node ever goes down, you can add a new node to the cluster and recover the lost partitions by copying data from one of the other nodes where copies of P1, P2, and P3 partitions are stored. Clearly, this is an extraordinarily complex maintenance process, but the Hadoop filesystem is a robust and time-tested framework. It can be reliable to 5 9s (99.999%).

Summary

In this video, you learned that:

- Big Data requires parallel processing because of capacity issues
- Parallel processing can be understood best by comparing it to Linear processing
- Parallel processing offers significant advantages over linear processing
- **Parallelism** in Big Data focuses on distributing the data across different nodes that operate on the data in parallel
- **Horizontal scaling**, or scaling out, means adding more machines or nodes to handle the increased workload, improving capacity and flexibility
- **Embarrassingly parallel** calculations easily divide workloads that run independent of one another. If any one process fails, it has no impact on the others and can simply be rerun
- **Fault tolerance** enables a system to continue operating properly in the event of the failure of some of its components